GenAI Frameworks for Python Developers

Master the tools and techniques for integrating Large Language Models into your Python applications. From API connections to prompt engineering, build powerful AI-powered features.

Made with GAMMA

Connecting Python to Modern LLMs

How It Works

Python communicates with LLMs through APIs or Software Development Kits (SDKs). The workflow is straightforward: write your code, send a prompt to the LLM API, the model processes your request, and you receive a response.

01

Write Python Code

Import libraries and initialize client

02

Send Prompt

Transmit your request to LLM API

03

Model Processing

LLM interprets and generates response

04

Receive Response

Process and use the output

Made with GAMMA

API Security Using Environment Variables

Why API Security Matters

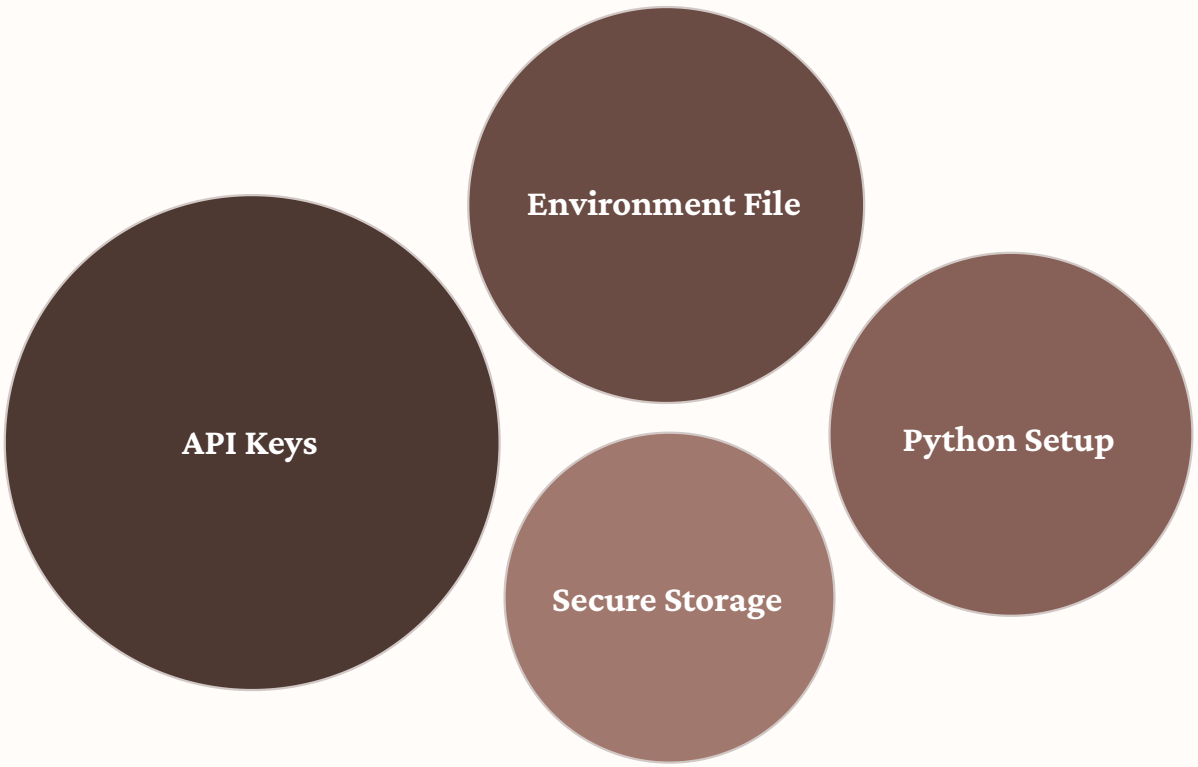
Never expose API keys directly in your code. Use environment variables to store secrets safely. This prevents unauthorized access to your AI platform accounts and maintains security best practices.

Example .env File

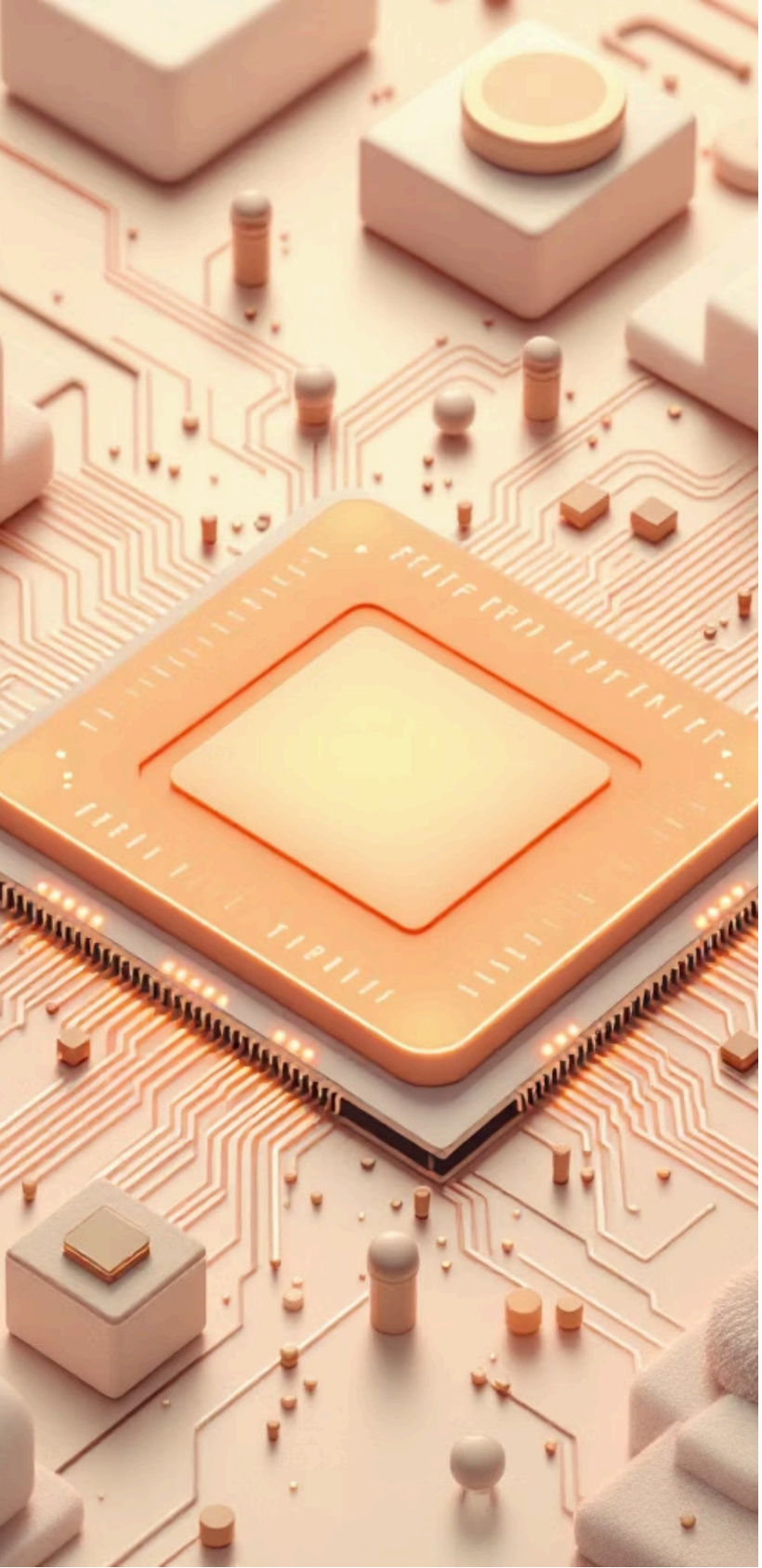
```
GROQ_API_KEY=your_secret_key
OPENAI_API_KEY=another_key
ANTHROPIC_API_KEY=third_key
```

Python Implementation

```
import os
from dotenv import load_dotenv

load_dotenv()
api_key = os.getenv("GROQ_API_KEY")
print(api_key)
```



Made with GAMMA



Groq: The Fast LLM Platform

Ultra-Fast Inference

Specialized hardware delivers exceptional speed

Low Latency

Minimal delay between request and response

Llama Support

Optimized for Llama models

Example Code

```
from langchain_groq import ChatGroq

llm = ChatGroq(
 model_name="llama-3.3-70b-versatile",
 temperature=0
)

print(llm.invoke("Define machine learning").content)
```

Made with GAMMA

Ollama: Run LLMs Locally

Benefits of Local LLMs

- No internet connection required
- Enhanced privacy and data security
- Faster iteration during development
- Reduced costs

Installation

Download and install Ollama from official website

Pull Models

Download Llama, Mistral, or other models

Query in Python

Use LangChain integration for easy access

Example Code

```
from langchain_community.llms import Ollama

llm = Ollama(model="llama3")
response = llm.invoke("Explain neural networks")
print(response)
```

Made with GAMMA

Python SDKs for AI Platforms

What is a Python SDK?

A Software Development Kit (SDK) is a library that simplifies API interactions. SDKs handle authentication, manage requests and responses, and provide ready-to-use functions, making development faster and more reliable.

Simplifies API Usage

Abstracts complex HTTP requests into simple function calls

Handles Authentication

Manages API keys and tokens automatically

Provides Ready Functions

Pre-built methods for common operations

Example: Groq SDK

```
from groq import Groq

client = Groq(api_key="API_KEY")
print(client)

# SDK handles requests, authentication, and responses
```

Made with GAMMA

Prompt Engineering Fundamentals

What is Prompting?

Prompting means giving clear instructions to an LLM to get the desired output. Well-structured prompts lead to better, more consistent results.

Good Prompt Structure

Instruction

What task to perform

Context

Background information

Input

Data to process

Output Format

Expected response structure

Example Prompt

```
prompt = """
Classify the sentiment of this text as
Positive, Negative, or Neutral.

Text: I love this phone.
"""
```

Made with GAMMA

Sentiment Analysis with LLMs

What is Sentiment Analysis?

Sentiment analysis identifies emotional tone in text, categorizing it as positive, negative, or neutral. This powerful technique helps applications understand user opinions, reviews, and feedback automatically.

Positive

Expresses approval, satisfaction, or enthusiasm

Negative

Shows disapproval, dissatisfaction, or criticism

Neutral

Contains factual information without emotion

Example Code

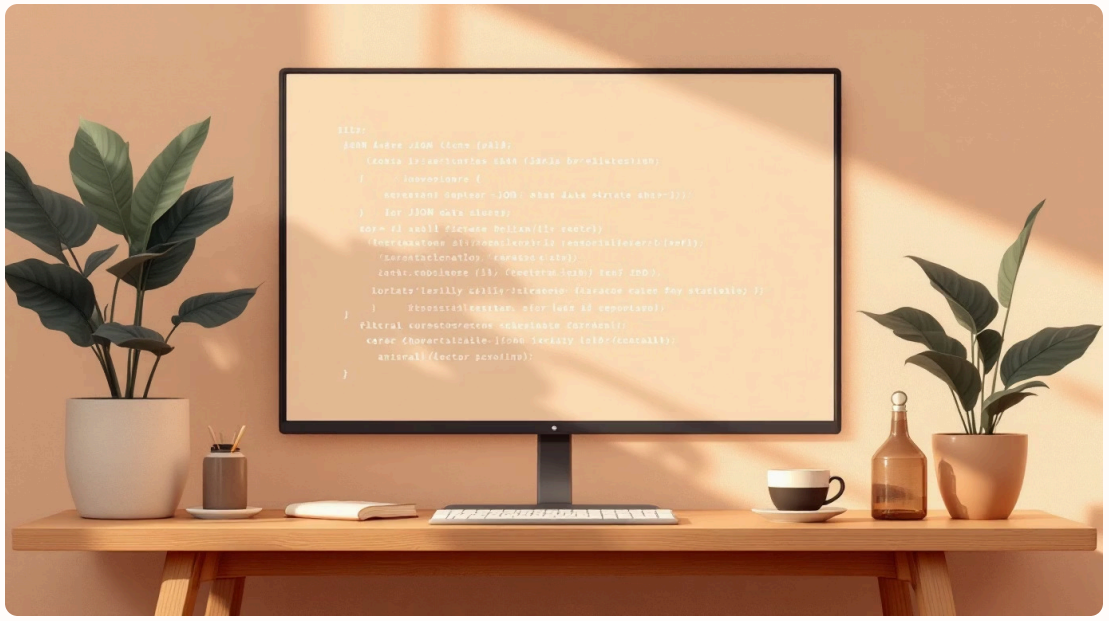
```
prompt = """
Classify the sentiment of this text.

Text: This product is amazing!
"""

print(llm.invoke(prompt).content)
# Output: Positive
```

Made with GAMMA

Structured Output with JSON

Why JSON Output?

Structured data is easier for applications to process programmatically. JSON format provides clear field names, consistent structure, and simple parsing with Python's built-in json module.



Example JSON Output

```
{
  "sentiment": "positive",
  "confidence": 0.95,
  "text": "I love this phone"
}
```

Python Implementation

```
import json

result = {
 "sentiment": "positive",
 "confidence": 0.95
}

print(json.dumps(result, indent=2))
```

Made with GAMMA

Comparing Sentiment Results

Experiment: Analyze Multiple Texts

Test your sentiment analysis model with various examples to understand its behavior and accuracy.

Example Code

```
texts = [
    "I love this laptop",
    "This phone is terrible",
    "The device has a 6 inch display"
]

for t in texts:
    prompt = f"Classify sentiment: {t}"
    result = llm.invoke(prompt).content
    print(f"{t} -> {result}")

# Expected Output:
# Positive
# Negative
# Neutral
```

Learning Outcome

Students understand prompt engineering, LLM inference, and AI text analysis

Next Steps

Experiment with different models and prompts to improve accuracy

Made with GAMMA